Linux Filesystem

01/10/2026 Aryan Singh

Introduction to Linux Filesystem

A Linux filesystem is the way Linux organizes and stores files, directories, devices, system information, and other resources. Unlike operating systems that organize storage using separate drive letters, Linux follows a single hierarchical directory structure that begins at the root directory '/' All other directories and files are arranged below this root in a tree-like structure.

Different storage devices and filesystems can be connected to the same overall directory structure through mount points. The filesystem provides a common structure through which users and programs can access data. For example, personal files are generally stored under '**/home**', system configuration files are commonly found under '**/etc**', temporary files can be stored in '**/tmp'**, and information about running processes and the kernel is exposed through pseudo-filesystems such as '**/proc'**.

One important concept in Linux is that many resources are represented through the filesystem interface. Devices are represented under '**/dev'**, kernel and device information is available through '**/sys'**, and process-related information is exposed through '**/proc'**. This makes interaction with many system resources possible using familiar filesystem operations and commands.

So, the Linux filesystem is not just a place where files are stored. It also provides a structured way for users and programs to interact with files, devices, and different parts of the operating system.

Key features of Linux Filesystem

1.Hierarchical Structure

The Linux filesystem follows a tree-like structure. The root directory '**/**' is at the top, and other directories branch out from it. This provides a consistent way of organizing and locating files.

2.Everything is organized through the filesystem

Linux exposes not only ordinary files and directories but also various system resources through filesystem interfaces. For example, '**`/dev`**' contains device files, while '**`/proc`**' and '**`/sys`**' provide interfaces to kernel and system information.

3.File Permissions and Ownership

Linux provides permissions to control who can access a file or directory. Every file has an owner and a group, along with permissions that determine whether it can be read, written, or executed. These permissions are associated with the owner, group, and other users. Commands such as '**`chmod`**' and '**`chown`**' can be used to modify permissions and ownership. This is particularly useful on systems where multiple users work on the same computer because it helps prevent unauthorized modification of files.

4. Different Types of Filesystems

Linux can work with different filesystem types. Examples include ext4 and XFS, while special filesystems such as '**`proc`**' and '**`sysfs`**' serve different purposes. This allows Linux to work with different types of storage and system interfaces rather than being limited to one filesystem format.

5.Mounting

Linux uses a concept called mounting to make a filesystem accessible at a particular location in the directory tree. Once mounted, its files can be accessed through that location. The '**`/mnt`**' directory is traditionally used as a mount point for temporarily mounted filesystems.

6.Symbolic Links

Linux supports symbolic links, commonly called symlinks. A symbolic link acts as a reference to another file or directory. This allows the same file or directory to be accessed through another path without creating another complete copy of the data.

7.Virtual and Pseudo-Filesystems

Some directories in Linux do not simply contain ordinary files stored on a disk. For example, '**`/proc`**' provides information about running processes and the kernel, while '**`/sys`**' provides structured information about devices and other kernel objects. This is one of the interesting aspects of Linux because system information can often be accessed using familiar filesystem operations.

8. Journaling

Journaling file systems maintain a special log called a journal that records file system changes before they are permanently written to disk. This helps restore data in case of unexpected shutdowns or crashes.

9.Versioning

Versioning file systems keep track of previous versions of a file, allowing users to recover older copies when needed.

10.Inode

An inode (index node) is a data structure that stores important information about files and directories. Inodes act as the internal identity of files in Linux file systems.

Directory Structure

The Linux directory structure starts from '**/**', which is called the root directory. Different directories under '**/**' have different purposes. Some important directories are:

Directory	Purpose
/	Root of the entire filesystem hierarchy
/bin	Essential executable programs
/boot	Files required for the boot process
/dev	Device files representing devices
/etc	System configuration files
/home	Personal directories of normal users
/lib	Essential shared libraries and kernel modules on traditional layouts
/media	Mount points commonly used for removable media

/mnt	Temporary mount point
/opt	Add-on software packages
/proc	Process and kernel information through a pseudo-filesystem
/root	Home directory of the root user
/run	Runtime system information since boot
/sbin	System administration executables on traditional layouts
/srv	Data served by the system
/sys	Kernel/device information through sysfs
/tmp	Temporary files
/usr	User-space programs, libraries, documentation and other data
/var	Variable data such as logs and caches

Basic Commands

Linux provides a wide range of commands for navigating the filesystem, creating and managing files, searching for information, handling permissions, and managing storage. The following commands cover the basic operations that are useful when working with the Linux filesystem.

Navigation Commands:

1. pwd – Print Working Directory
The pwd command displays the absolute path of the directory currently being used. It is useful when we want to know exactly where we are in the filesystem.

2. ls – List Files and Directories
The ls command displays the files and directories present in the current location.

ls
Lists the contents of the current directory.
ls -l
Shows detailed information such as permissions, owner, size and modification time.

ls -a
Shows all files, including hidden files whose names begin with ..
ls -lh
Displays detailed information with file sizes in a human-readable format such as KB or MB.

3. cd – Change Directory
The cd command is used to move from one directory to another.

cd
Moves to the user's home directory.
cd ..
Moves to the parent directory.
cd /path
Moves directly to the specified directory.

File & Directory Management :

4. mkdir – Make Directory
The mkdir command creates a new directory.

mkdir
Creates a directory.
mkdir -p
Creates the complete directory structure if the parent directories do not already exist.

5. rmdir – Remove Empty Directory
The rmdir command removes a directory only if it is empty. It is useful for safely removing directories that no longer contain files.

6. touch – Create a File
The touch command can create a new empty file. It can also update the modification time of an existing file.

7. cp – Copy Files
cp
The cp command is used to make a copy of a file.
cp -r
The -r option allows directories and their contents to be copied recursively.

8. mv – Move or Rename
The mv command is used to move files/directories or rename them.

9. rm – Remove Files

rm
The rm command removes files.
rm -r
Removes a directory and its contents recursively.

Viewing Files:

10. cat – Display File Contents
The cat command displays the contents of a text file. It is useful for quickly viewing small text files.

11. head – View Beginning of a File

head
The head command displays the beginning of a file. By default, it displays the first few lines.
head -n
Displays the first 5 lines of the file.

12. tail – View End of a File

tail
The tail command displays the end of a file.
tail -n
Displays the last 5 lines. It is particularly useful for checking the latest entries in log files.

13. less – View Large Files
The less command allows a file to be viewed page by page. It is useful for large files because the entire file does not have to be displayed at once.

Searching :

14. find – Search for Files
The find command searches for files and directories based on different conditions.

Search by name
Searches the current directory and its subdirectories for files ending in name we searched.
Search for a directory
Displays directories found under the current location.

15. grep – Search Text
The grep command searches for a particular pattern or word inside text.

grep
Displays lines containing the word we searched.
grep -i
Performs a case-insensitive search.
grep -n
Displays the line number along with matching lines.

Useful File Processing :

16. wc – Count Lines, Words and Characters

wc
The wc command provides counts related to a file.
wc -l
Counts the number of lines.
wc -w
Counts the number of words.

17. sort – Sort Text
The sort command arranges lines in a particular order.

sort
By default, it sorts the lines alphabetically.
sort -r
Sorts the lines in reverse order.

18. uniq – Remove Repeated Lines
The uniq command removes repeated adjacent lines.

19. cut – Extract Parts of Text
The cut command extracts selected sections from each line of text.

20. diff – Compare Files
The diff command compares two text files and shows their differences. It can be useful when checking changes between two versions of a file.

Permissions & Ownership :

21. chmod – Change File Permissions
The chmod command changes the permissions of a file or directory.

22. chown – Change File Ownership
The chown command changes the owner of a file or directory. The command generally requires appropriate privileges when changing ownership.

23. chgrp – Change Group Ownership
The chgrp command changes the group associated with a file. This is useful when files need to be shared with members of a particular group.

24. umask – Set Default Permission Mask
The umask command controls the permissions that are normally removed from newly created files and directories.

umask
Displays the current mask.
The exact resulting permissions also depend on the permissions requested by the program creating the file.

Links :

25. ln – Create Links
The ln command creates links to files.

Hard link
Creates a hard link to the original file.

Symbolic link
Creates a symbolic link that points to the original file. Symbolic links are commonly used as shortcuts to files or directories.

File Information & Paths :

26. file – Identify File Type
The file command determines the type of a file. It can identify whether a file is a text file, executable, image, archive, etc.

27. stat – Display File Information

The stat command displays detailed information about a file. It can show information such as file size, permissions, ownership, inode number, access time, modification time, etc.

28. basename – Extract File Name
The basename command extracts the final component of a path.

29. dirname – Extract Directory Path
The dirname command extracts the directory portion of a path.

30. tree – Display Directory Structure
The tree command displays directories and files in a tree-like structure. It is useful for getting a quick visual overview of a directory structure.

Storage & Filesystems :

31. df – Check Filesystem Disk Space

df
The df command displays information about available and used disk space on mounted filesystems.
df -h
The -h option displays sizes in a human-readable format.

32. du – Check Directory/File Size

du
The du command shows how much disk space files and directories are using.
du -sh
Shows the total size of a directory in a human-readable format.

33. mount / umount – Manage Mounted Filesystems

mount
The mount command is used to attach a filesystem to a directory in the Linux filesystem hierarchy. It can also be used with appropriate privileges and options to mount a filesystem at a specified location.

umount
The umount command detaches a mounted filesystem.

Storage & Filesystems :

34. tar – Archive Files
The tar command is commonly used to combine multiple files and directories into an archive. It can also be used to extract an archive.

35. gzip – Compress Files
The gzip command compresses files to reduce their size.
gunzip
A compressed file can be decompressed.

References

Opensource.com - https://opensource.com/life/16/10/introduction-linux-filesystems
GeeksforGeeks - https://www.geeksforgeeks.org/linux-unix/linux-file-system/
Linux Foundation - https://www.linuxfoundation.org/blog/blog/classic-sysadmin-the-linux-filesystem-explained
Dev Community - https://dev.to/prodevopsguytech/understanding-the-linux-filesystem-an-in-depth-guide-for-devops-engineers-ona
Medium.com - https://medium.com/@subhampradhan966/linux-fundamentals-and-basic-commands-30a1ea72ea11
Linuxhandbook.com - https://linuxhandbook.com/linux-directory-structure/

Review Questions

Report 2:

Introduction to Version Control

Version control is a system which keeps track and helps to manage all the changes made to files over time. It is especially useful when working on software projects, where programmers need to continuously modify their codes.

One of the most widely used version control systems today is Git. It allows developers to save different versions of their projects, compare changes, and a lot more.

Version Control Systems work using a few core concepts that help teams manage code changes and collaborate efficiently.

- ➢ Repository: A central location that stores all project files along with their complete change history and data like author and commit message.
- ➢ Revision: A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- ➢ Branch: A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- ➢ Merging: The process of combining changes from one branch into another
- ➢ Commit: A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

In short, version control is like the history of a project. It tells us what was changed, who changed it, when the change was made and combine the changes, making the work way easier when working in a team.

Importance of Version Control

Version control is important due to many reasons like :

a) It keeps history of changes made

Version control records the changes made to a project. This is very useful as if a programmer changes a section of code today and the program stops working, they can check the previous version, identify what was changed and figure out the error. It also is very helpful, if a developer accidentally removes something, as it can be recovered through the previous versions. And in larger projects since it records information about changes made, including the people who made them. This makes it easier for a team to understand the development history of a project.

b) Makes Teamwork Easier

In a group project, several people may need to work on the same code. Hence, version control allows everyone to create separate branches for their work, make their own changes and later combine them into the main project. This is very very useful for large software projects where many developers are working simultaneously.

c) Supports Experimentation

Developers can create a separate branch and experiment with a new feature without immediately affecting the main version of the project. If the experiment works, it can later be merged into the main branch. Therefore, version control is useful even for safely managing changes, trying new features and improving the project, without affecting the live version instantaneously.

Types of Version Control

Version control systems can broadly be divided into centralized and distributed systems. Local version control can also be considered an earlier/basic approach to managing versions.

Local Version Control

In a local version control system, different versions of files are maintained on the same computer. This approach can work for a single person and a small project, but it has limitations as there is no convenient system for collaboration, and losing the computer or its storage could result in losing the project history. Local version control therefore provides basic version tracking but is not ideal for larger team-based projects.

Centralized Version Control

In a Centralized Version Control System (CVCS), a central server stores the project's files and version history. Developers connect to this central repository to obtain the project and submit their changes.

For example, Subversion (SVN) and Perforce.

The main advantage of this approach is that everyone works with a common central repository. However, the central server becomes an important part of the workflow. If it is unavailable, some version control operations and collaboration may be affected.

Distributed Version Control

In a Distributed Version Control System (DVCS), each developer has a complete copy of the repository, including its history.

Git and Mercurial are examples of distributed version control systems.

This approach provides more flexibility because many operations can be performed locally without continuously depending on a central server. Developers can make commits, examine history, and work with branches on their own computers. Changes can later be shared with other repositories.

Apart from these, there are some specialised models of Version Control like:

Lock Based Version Control

A lock-based version control system uses file locking to manage concurrent access to files and resources. File locking prevents two or more users from making conflicting changes to the same file or resource in large binary or proprietary assets, which comprises of complex data, not represented as plain text, in terms of rovers are 3D models of rover, high resolution maps, sensor logs, etc.

Semantic / Lineage-Based Version Control

It helps to train an AI model, for example in terms of rovers if we need to train a rover on 50 gigabytes of terrain photos so the rover can recognize rocks.
These massive datasets, video logs, or trained model weights will be stored on a dedicated storage server or cloud bucket like AWS S3.

The semantic versioning tool (like DVC) creates a tiny, human-readable text file for example, “terrain_dataset.dvc” containing a unique cryptographic hash of that massive dataset.
When a teammate goes to an older code branch, the pointer changes, telling the system to automatically download the exact version of the massive dataset that matched that specific historical moment of the code.

Important Git Commands

Git is a distributed version control system used to track changes in files and collaborate with other team members. In a project such as a rover project, Git can help multiple members work on different parts of the code while maintaining a history of changes.

The following are some important Git commands and their commonly used variations.

1. git --version

Used to check the installed Git version.

Common usages:

- git --version – Displays the installed Git version.

2. git config

Used to view and modify Git configuration settings.

Common usages:

- git config --global user.name "Your Name" – Sets the Git username.
- git config --global user.email "email" – Sets the Git email.
- git config --list – Displays Git configuration settings.
- git config --get user.name – Displays the configured username.
- git config --unset user.name – Removes the configured username.

3. git init

Used to create a new Git repository.

Common usages:

- git init – Initializes the current directory as a Git repository.
- git init <directory> – Initializes a repository in the specified directory.

4. git clone

Used to create a local copy of a remote repository.

Common usages:

- git clone <repository-url> – Downloads a repository to the local system.
- git clone <repository-url> <directory> – Clones it into a specified directory.

5. git status

Used to check the current state of the repository.

Common usages:

- git status – Shows modified, staged, and untracked files.
- git status -s – Shows the status in a shorter format.

6. git add

Used to move changes to the staging area.

Common usages:

- git add <file> – Stages a specific file.
- git add . – Stages changes in the current directory.
- git add -A – Stages all changes.

7. git commit

Used to save staged changes in the repository history.

Common usages:

- git commit -m "message" – Creates a commit with a message.
- git commit -am "message" – Stages tracked-file changes and commits them.
- git commit --amend – Modifies the latest commit.

8. git diff

Used to view differences between file versions.

Common usages:

- git diff – Shows unstaged changes.
- git diff --staged – Shows staged changes.
- git diff <commit1> <commit2> – Compares two commits.

9. git log

Used to view commit history.

Common usages:

- git log – Shows detailed commit history.
- git log --oneline – Shows commits in a compact format.
- git log --graph --oneline – Shows commit history with a branch graph.

10. git show

Used to view information about a commit.

Common usages:

- git show <commit> – Shows the commit and its changes.
- git show --stat <commit> – Shows a summary of changed files.

11. git restore

Used to restore files or remove them from the staging area.

Common usages:

- git restore <file> – Discards unstaged changes.
- git restore --staged <file> – Removes a file from staging.
- git restore --source <commit> <file> – Restores a file from a specific commit.

12. git reset

Used to move HEAD to another commit.

Common usages:

- git reset --soft <commit> – Moves HEAD while keeping changes staged.
- git reset --mixed <commit> – Moves HEAD and unstages changes.
- git reset --hard <commit> – Resets HEAD, staging area, and working files.

13. git revert

Used to undo a previous commit without removing it from history.

Common usages:

- git revert <commit> – Creates a new commit that reverses the specified commit.
- git revert --no-edit <commit> – Reverts without editing the commit message.

14. git branch

Used to create and manage branches.

Common usages:

- git branch – Lists local branches.
- git branch <branch> – Creates a new branch.
- git branch -a – Lists local and remote branches.
- git branch -d <branch> – Deletes a merged branch.

15. git switch

Used to switch between branches.

Common usages:

- git switch <branch> – Switches to a branch.
- git switch -c <branch> – Creates and switches to a new branch.
- git switch - – Switches to the previous branch.

16. git merge

Used to combine changes from different branches.

Common usages:

- git merge <branch> – Merges the specified branch into the current branch.
- git merge --abort – Cancels an ongoing merge.

17. git remote

Used to manage remote repositories.

Common usages:

- git remote -v – Shows remote repository URLs.
- git remote add origin <url> – Adds a remote repository.
- git remote remove origin – Removes a remote connection.

18. git fetch

Used to download changes from a remote repository without merging them.

Common usages:

- git fetch – Downloads remote updates.
- git fetch --all – Fetches updates from all remotes.
- git fetch --prune – Removes outdated remote-tracking references.

19. git pull

Used to download and integrate changes from a remote repository.

Common usages:

- git pull – Fetches and integrates remote changes.
- git pull --rebase – Fetches changes and integrates them using rebase.
- git pull origin main – Pulls changes from the remote main branch.

20. git push

Used to upload local commits to a remote repository.

Common usages:

- git push – Uploads local commits.
- git push -u origin main – Pushes main and sets its upstream.
- git push --delete origin <branch> – Deletes a remote branch.

21. git stash

Used to temporarily save uncommitted changes.

Common usages:

- git stash – Saves current changes temporarily.
- git stash pop – Restores the latest stash.
- git stash list – Shows saved stashes.
- git stash apply – Restores a stash without removing it.

22. git tag

Used to mark specific commits, usually for versions or releases.

Common usages:

- git tag – Lists tags.
- git tag <tag-name> – Creates a tag.
- git tag -a <tag-name> -m "message" – Creates an annotated tag.

23. git mv

Used to move or rename tracked files.

Common usages:

- git mv <old-name> <new-name> – Renames or moves a file.

24. git rm

Used to remove files from the repository.

Common usages:

- git rm <file> – Removes and stages deletion of a file.
- git rm --cached <file> – Stops tracking a file but keeps it locally.
- git rm -r <directory> – Removes a directory and its contents.

25. git clean

Used to remove untracked files.

Common usages:

- git clean -n – Shows files that would be removed.
- git clean -f – Removes untracked files.
- git clean -fd – Removes untracked files and directories.

26. git rebase

Used to reapply commits on top of another branch.

Common usages:

- git rebase <branch> – Rebases the current branch onto another branch.
- git rebase --continue – Continues a rebase after resolving conflicts.
- git rebase --abort – Cancels the rebase.

27. git cherry-pick

Used to apply a specific commit to the current branch.

Common usages:

- git cherry-pick <commit> – Applies the specified commit.
- git cherry-pick --continue – Continues after resolving conflicts.
- git cherry-pick --abort – Cancels the operation.

28. git reflog

Used to view changes made to HEAD and other references.

Common usages:

- git reflog – Shows the reference history.
- git reflog show <branch> – Shows the reference history of a branch.

29. git blame

Used to identify who last modified each line of a file.

Common usages:

- git blame <file> – Shows the author and commit for each line.
- git blame -L <start>,<end> <file> – Shows information for a specific range of lines.

30. git bisect

Used to find the commit that introduced a bug.

Common usages:

- git bisect start – Starts the bisect process.
- git bisect good <commit> – Marks a commit as working correctly.
- git bisect bad <commit> – Marks a commit as faulty.
- git bisect reset – Ends the bisect process.

References

https://www.atlassian.com/git/tutorials/what-is-version-control

https://about.gitlab.com/topics/version-control/

https://www.geeksforgeeks.org/git/version-control-systems/

https://senertugrul.medium.com/version-control-3f22d1bdeb52

https://dev.to/prodevopsguytech/github-30-github-commands-used-by-every-devops-engineer-4llj

https://docs.gitlab.com/topics/git/commands/

Review Questions

Review Questions – Sensors on a Rover

1. What is the difference between a 6-DOF and 9-DOF IMU?

Answer. An IMU (Inertial Measurement Unit) is a sensor used to measure the motion and orientation of a rover.

A 6-DOF IMU normally consists of:

- A 3-axis accelerometer, which measures linear acceleration along the X, Y and Z axes.
- A 3-axis gyroscope, which measures angular velocity, or rotation, about the X, Y and Z axes.

A 9-DOF IMU contains these same two sensors along with a 3-axis magnetometer, which measures the Earth's magnetic field along the three axes. The magnetometer gives additional information about the Earth's magnetic field and can be used to estimate the rover's heading with respect to magnetic north.

2. What are the 3 degrees of freedom with respect to the magnetic component of an IMU?

Answer. The magnetic component of a 9-DOF IMU is the 3-axis magnetometer. It measures the magnetic field along three perpendicular axes:

- X-axis – magnetic field in the X direction
- Y-axis – magnetic field in the Y direction
- Z-axis – magnetic field in the Z direction

Together, these three measurements describe the magnetic-field vector around the rover. The magnetometer is particularly useful for determining the rover's heading or yaw with respect to the Earth's magnetic field.

3. Encoder vs IMU – What is better for use on the arm?

Answer. An encoder measures the position or rotation of a motor or joint. This makes it very useful for controlling the exact movement of a robotic arm. An IMU measures acceleration, angular velocity and orientation. It can be used when information about the overall motion or

orientation of the arm is required. For example, if a robotic arm has to rotate a joint to exactly 90°, an encoder can provide feedback about how much the joint has rotated. Therefore, for a rover's robotic arm, an encoder is generally more suitable for precise joint-position control, while an IMU is useful for measuring the orientation and motion of the arm. In some systems, both sensors can be used together.

4. Why is a stereo camera digital?

Answer. A stereo camera is not necessarily digital. The term stereo refers to the use of two cameras or two viewpoints to obtain depth information. The two cameras capture the same scene from slightly different positions. The difference between the two images is called disparity, and it can be used to calculate the distance or depth of objects using triangulation. Most modern stereo camera systems are digital because digital images can be easily transferred to a computer or processor for image processing and depth calculation.

5. Where would you use an analog camera vs a digital camera?

Answer. The choice between an analog and digital camera depends mainly on the requirements of the application. An analog camera produces a continuous video signal. It can be useful for applications where simple real-time video transmission is required, especially with older or legacy systems. For example, it can be used for a basic video feed where advanced image processing is not required. A digital camera converts the captured image into digital data. This makes it much more suitable for computer-based image processing. Digital cameras are commonly used for Computer vision, Object detection, Machine learning, Stereo vision, Image storage, Autonomous navigation For a Mars rover, a digital camera is generally more useful because the captured images can be processed by the onboard computer for tasks such as obstacle detection, terrain analysis, object identification and autonomous navigation.

In simple terms:

Analog camera - mainly useful for basic video transmission

Digital camera - useful when the image needs to be processed by a computer

6. What is the difference between git reset and git revert?

Answer: git reset moves the current branch or HEAD to another commit, with its effect depending on the mode used. git revert creates a new commit that reverses the changes introduced by an earlier commit.

In simple terms:
git reset - moves the branch/HEAD
git revert - creates a new undo commit

7. What is the difference between git fetch and git pull?

Answer: git fetch downloads information and changes from a remote repository without integrating them into the current branch. git pull fetches the changes and then integrates them into the current branch.
In simple terms:
git fetch - download and inspect
git pull - download and integrate

8. How many commands are there in git config?

Answer: git config is one main Git command with multiple operations, options, and configuration scopes. It does not have a fixed number of completely separate commands.
Some commonly used usages are:
git config --list
git config --get user.name
git config --unset user.name
git config --edit
Common configuration scopes include:
--system
--global
--local
--worktree